CS 4644/7643: Deep Learning
Fall 2026
Problem Set 1

Instructor: Danfei Xu
TAs: Rohan Bansal, Uzu (Chris) Lee, Alfred Cueva
Kun-Lin Hsieh, Chuye Zhang, Liqian Ma, Robert Azarcon
Dhruv Ahuja, Xinchen Yin
Discussions: https://piazza.com/class/ms43b2tjxa63lq#

September 4, 2026

Student Name: Venkata Aashutosh Aripirala **GT Email:** av84@gatech.edu **GT ID:** 904173725

Instructions

1. We will be using Gradescope to collect your assignments. Please read the following instructions for submitting to Gradescope carefully!

   - For the **HW1 Theory** component on Gradescope, upload one single PDF containing the answers to all the theory questions. **However, the solution to each problem/subproblem must be on a separate page. When submitting to Gradescope, please make sure to mark the page(s) corresponding to each problem/subproblem.**
   - For the **HW1 Coding** component on Gradescope, please use the `collect_submission.py` script provided and upload the resulting **hw1_code_submission.zip** on Gradescope. Please make sure you have saved the most recent version of your code.
   - For the **HW1 Notebook** component on Gradescope, please use the `collect_submission.py` script provided and upload the resulting **hw1_notebook_submission.pdf** on Gradescope.
   - Note: This is a large class and Gradescope's assignment segmentation features are essential. Failure to follow these instructions may result in parts of your assignment not being graded. We will not entertain regrading requests for failure to follow instructions.

2. LaTeX'd solutions are strongly encouraged (solution template is provided in hw1_overleaf.zip), but scanned handwritten copies are acceptable. Hard copies are **not** accepted.

3. We generally encourage you to collaborate with other students.

   You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and *not* as a group activity. Please list the students you collaborated with.

1 Collaborators [0.5 points]

Please list your collaborators and assign this list to the corresponding section of the outline on Gradescope. If you don't have any collaborators, please write 'None' and assign it to the corresponding section of the Gradescope submission regardless. Not assigning this question on Gradescope or leaving it blank will be marked as zero.

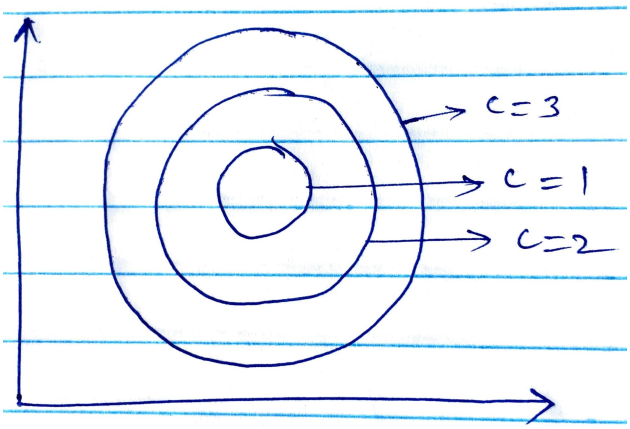
Solution: None

2 Optimization

1. **[3 points]** Let $f(\mathbf{x})$ be a function, where $\mathbf{x} \in \mathbb{R}^d$. The *level surface* of a function is the set of points with the same value of function, *i.e.*

$$L_c = \{\mathbf{x} \mid f(\mathbf{x}) = c\} \tag{1}$$

for some c $\in \mathbb{R}$.



The above figure shows level surfaces for different values of c for a 2-dimensional case. The points on the perimeter of each curve illustrate the set of points sharing the same value, i.e. $f(x_1, x_2) = c$.

Recall that a gradient vector is defined as $\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_d}\right]$.

Consider a specific point $\mathbf{x}_0 \in \mathbb{R}^d$. Let us denote the level surface passing through this point by $L_{f(\mathbf{x_0})}$. Let us denote the gradient vector at $\mathbf{x_0}$ by ∇f_0.

Now, consider an arbitrary curve $\mathbf{r}(t) = [x_1(t); x_2(t); \ldots, x_d(t)]$ (parameterized by a scalar t) that passes through $\mathbf{x}_0$, *i.e.* $\mathbf{r}(t_0) = \mathbf{x_0}$ for some $t_0 \in \mathbb{R}$. Furthermore, the curve $\mathbf{r}(t)$ lies within the level surface passing through $\mathbf{x}_0$, *i.e.* $\mathbf{r}(t) \in L_{f(\mathbf{x_0})}$, $\forall t$.

Recall that the *tangent* to a curve is defined as $\frac{\partial \mathbf{r}}{\partial t}$.

With all that context and notation behind us, now comes the 'fun' part – show that ∇f_0 is orthogonal to the tangent of $\mathbf{r}(t)$ at t_0.

Now, please describe in non-technical language what you have just proven. Why might we care about this in the context of deep learning?

Solution: 1. Proof of Orthogonality:

Since the curve $\mathbf{r}(t)$ is entirely contained within the level surface $L_{f(\mathbf{x}_0)}$ for all t, the value of the function f evaluated along the trajectory of the curve is constant:

$$f(\mathbf{r}(t)) = c = f(\mathbf{x}_0) \quad \forall t \tag{2}$$

Differentiating both sides with respect to t:

$$\frac{\mathrm{d}}{\mathrm{d}t}\left[f(\mathbf{r}(t))\right] = \frac{\mathrm{d}}{\mathrm{d}t}[c] = 0 \tag{3}$$

Applying the chain rule to LHS:

$$\frac{\mathrm{d}}{\mathrm{d}t}\left[f(\mathbf{r}(t))\right] = \sum_{i=1}^{d} \frac{\partial f}{\partial x_i}(\mathbf{r}(t)) \frac{\mathrm{d}x_i}{\mathrm{d}t}(t) = \nabla f(\mathbf{r}(t)) \cdot \frac{\mathrm{d}\mathbf{r}}{\mathrm{d}t}(t) = 0 \tag{4}$$

Evaluating this at $t = t_0$, where $\mathbf{r}(t_0) = \mathbf{x}_0$ and $\nabla f(\mathbf{r}(t_0)) = \nabla f(\mathbf{x}_0) = \nabla f_0$:

$$\nabla f_0 \cdot \frac{\mathrm{d}\mathbf{r}}{\mathrm{d}t}(t_0) = 0 \tag{5}$$

Since the dot product between the gradient vector ∇f_0 and the tangent vector $\frac{\mathrm{d}\mathbf{r}}{\mathrm{d}t}(t_0)$ is zero, ∇f_0 is orthogonal to the tangent vector of any curve lying on the level surface at $\mathbf{x}_0$.

2. Non-Technical Interpretation & Importance in Deep Learning:

- **Non-Technical Interpretation:** A level surface (analogous to a contour line on a topographical elevation map) connects all points that share the exact same height. Moving along a level surface yields zero rate of change in the function. The proof demonstrates that the gradient vector points in a direction that is strictly perpendicular to these contour lines. Geometrically, this means the gradient points directly away from the path of zero change, pointing along the direction of steepest ascent.
- **Why We Care in Deep Learning:** In deep learning, training a neural network corresponds to minimizing a high-dimensional loss represented as a surface $L(\mathbf{W})$, a function of the model weights $\mathbf{W}$. Because the gradient $\nabla L(\mathbf{W})$ is perpendicular to the contours of equal loss, the negative gradient that we compute in Gradient Descent etc. $-\nabla L(\mathbf{W})$ points in the direction of *steepest descent*. First-order optimization algorithms such as Gradient Descent, SGD, Momentum, and Adam rely fundamentally on this geometric property to efficiently navigate across the loss surface to arrive at the global minima.

2. **[2 points]**

The softmax function $\mathbf{s}(\mathbf{z})$, which takes a vector input $\mathbf{z}$ and outputs a vector whose ith entry s_i is

$$s_i = \frac{e^{z_i}}{\sum_k e^{z_k}} \tag{6}$$

The input vector $\mathbf{z}$ to $\mathbf{s}(\cdot)$ is sometimes called the "logits", which just means the unscaled output of previous layers. Derive the gradient of $\mathbf{s}$ with respect to the logits, *i.e.* derive $\frac{\partial \mathbf{s}}{\partial \mathbf{z}}$.

Solution: Let $\mathbf{z} = [z_1, z_2, \ldots, z_K]^T \in \mathbb{R}^K$ and let $\Sigma = \sum_{k=1}^K e^{z_k}$. The i-th component of the softmax function is $s_i = \frac{e^{z_i}}{\Sigma}$.

We want to compute the Jacobian matrix $\frac{\partial \mathbf{s}}{\partial \mathbf{z}} \in \mathbb{R}^{K \times K}$, whose (i,j)-th entry is $\frac{\partial s_i}{\partial z_j}$. Applying the quotient rule:

$$\frac{\partial s_i}{\partial z_j} = \frac{\frac{\partial (e^{z_i})}{\partial z_j}\Sigma - e^{z_i}\frac{\partial \Sigma}{\partial z_j}}{\Sigma^2} \tag{7}$$

Evaluating partial derivatives involved:

$$\frac{\partial (e^{z_i})}{\partial z_j} = \begin{cases} e^{z_i} & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} = \delta_{ij} e^{z_i} \tag{8}$$

$$\frac{\partial \Sigma}{\partial z_j} = \frac{\partial}{\partial z_j}\left(\sum_{k=1}^K e^{z_k}\right) = e^{z_j} \tag{9}$$

where $\delta_{ij} = 1$ if $i = j$, and $\delta_{ij} = 0$ if $i \neq j$.

Substituting these back into the quotient rule:

$$\frac{\partial s_i}{\partial z_j} = \frac{\delta_{ij} e^{z_i}\Sigma - e^{z_i}e^{z_j}}{\Sigma^2} \tag{10}$$

$$= \delta_{ij}\frac{e^{z_i}}{\Sigma} - \left(\frac{e^{z_i}}{\Sigma}\right)\left(\frac{e^{z_j}}{\Sigma}\right) \tag{11}$$

$$= \delta_{ij} s_i - s_i s_j \tag{12}$$

$$= s_i(\delta_{ij} - s_j) \tag{13}$$

$$\frac{\partial s_i}{\partial z_j} = \begin{cases} s_i(1 - s_i) & \text{if } i = j \\ -s_i s_j & \text{if } i \neq j \end{cases} \tag{14}$$

The Jacobian matrix is given by:

$$\frac{\partial \mathbf{s}}{\partial \mathbf{z}} = \text{diag}(\mathbf{s}) - \mathbf{s}\mathbf{s}^T \tag{15}$$

where $\text{diag}(\mathbf{s})$ is the $K \times K$ diagonal matrix containing vector $\mathbf{s}$ on its diagonal.

3. **[3 points: Extra credit for both 4644 and 7643]**

Recall that a $(d-1)$-dimensional simplex is defined as:

$$\Delta_{d-1} = \{\mathbf{p} \in \mathbb{R}^d \mid \mathbf{1}^T\mathbf{p} = 1 \text{ and } p_i \geq 0 \ \forall i \in \{1, \ldots, d\}\} \tag{16}$$

In this question, you will develop an interpretation of softmax as a projection operator – that it projects an arbitrary point $\mathbf{x} \in \mathbb{R}^d$ onto the interior of the $d-1$ simplex. Specifically, let $\mathbf{s}(\mathbf{x})$ denote the softmax function (as defined above). Now prove that,

$$\mathbf{s}(\mathbf{x}) = \underset{\mathbf{y}\in\mathbb{R}^d}{\operatorname{argmin}} \quad -\mathbf{x}^T\mathbf{y} - H(\mathbf{y}) \tag{17}$$

$$s.t. \quad \mathbf{1}^T\mathbf{y} = 1, \quad 0 \leq y_i \leq 1 \quad \forall i \tag{18}$$

where H is the entropy function:

$$H(\mathbf{y}) = -\sum_i y_i \log(y_i) \tag{19}$$

Now, what does this formal interpretation tell you about the softmax layer in a neural network? Hint: Look at the KKT conditions.

Solution: 1. Proof via KKT Conditions:

We consider the constrained optimization problem:

$$\min_{\mathbf{y}\in\mathbb{R}^d} f(\mathbf{y}) = -\mathbf{x}^T\mathbf{y} - H(\mathbf{y}) = -\sum_{i=1}^{d} x_i y_i + \sum_{i=1}^{d} y_i \ln y_i \tag{20}$$

subject to:

$$h(\mathbf{y}) = \sum_{i=1}^{d} y_i - 1 = 0 \tag{21}$$

$$g_{1,i}(\mathbf{y}) = -y_i \leq 0, \quad \forall i \in \{1, \ldots, d\} \tag{22}$$

$$g_{2,i}(\mathbf{y}) = y_i - 1 \leq 0, \quad \forall i \in \{1, \ldots, d\} \tag{23}$$

The objective function $f(\mathbf{y})$ is strictly convex on the interior of the simplex because the negative entropy function $-H(\mathbf{y}) = \sum_i y_i \ln y_i$ is strictly convex on $(0, 1]^d$ (its Hessian $\nabla^2(-H(\mathbf{y})) = \text{diag}(1/y_1, \ldots, 1/y_d)$ is strictly positive definite for $y_i > 0$), and $-\mathbf{x}^T\mathbf{y}$ is linear. Since the constraints are affine, Slater's condition is satisfied, and the KKT conditions are both necessary and sufficient for the unique global optimum $\mathbf{y}^*$.

Furthermore, $\lim_{y_i\to 0^+} \frac{\mathrm{d}}{\mathrm{d}y_i}(y_i \ln y_i) = \lim_{y_i\to 0^+}(\ln y_i + 1) = -\infty$. Consequently, any boundary point where $y_i = 0$ would cause the directional derivative outward from the boundary to be $-\infty$, pulling the solution strictly into the interior $y_i > 0$. Therefore, the optimal solution lies strictly in the relative interior of the simplex: $0 < y_i < 1$ for all i.

By complementary slackness, the inequality multipliers associated with $-y_i \leq 0$ and $y_i \leq 1$ are zero ($\mu_i = 0, \nu_i = 0, \forall i$).

The Lagrangian with equality multiplier $\lambda \in \mathbb{R}$ is:

$$\mathcal{L}(\mathbf{y}, \lambda) = \sum_{i=1}^{d} (y_i \ln y_i - x_i y_i) + \lambda \left(\sum_{i=1}^{d} y_i - 1 \right) \tag{24}$$

Setting the partial derivative of $\mathcal{L}$ with respect to each y_i to zero (stationarity condition):

$$\frac{\partial \mathcal{L}}{\partial y_i} = \ln y_i + 1 - x_i + \lambda = 0 \tag{25}$$

$$\ln y_i = x_i - 1 - \lambda \tag{26}$$

$$y_i = e^{x_i - 1 - \lambda} = e^{-(1+\lambda)} e^{x_i} \tag{27}$$

Since $\sum_{i=1}^{d} y_i = 1$:

$$\sum_{i=1}^{d} e^{-(1+\lambda)} e^{x_i} = e^{-(1+\lambda)} \sum_{i=1}^{d} e^{x_i} = 1 \implies e^{-(1+\lambda)} = \frac{1}{\sum_{k=1}^{d} e^{x_k}} \tag{28}$$

Substituting $e^{-(1+\lambda)}$ back into the equation for y_i:

$$y_i^* = \frac{e^{x_i}}{\sum_{k=1}^{d} e^{x_k}} = s_i(\mathbf{x}) \tag{29}$$

Thus, the unique minimizer is the softmax output $\mathbf{y}^* = \mathbf{s}(\mathbf{x})$.

2. Formal Interpretation for Neural Networks:

- **Entropy-Regularized Maximum Linear Alignment:** Without the entropy term $-H(\mathbf{y})$, the optimization problem $\min_{\mathbf{y} \in \Delta} -\mathbf{x}^T \mathbf{y} \iff \max_{\mathbf{y} \in \Delta} \mathbf{x}^T \mathbf{y}$ corresponds to the standard hard argmax operation, which places probability 1 on $\operatorname{argmax}_i x_i$ and 0 elsewhere. The hard argmax is a discontinuous, non-differentiable step function.
- **Smooth Differentiable Approximation:** Adding the negative entropy penalty $-H(\mathbf{y})$ acts as a regularizer that maximizes entropy (encouraging uncertainty and spreading the probability mass function) while aligning with the logits $\mathbf{x}$. This turns the hard projection onto the simplex into a smooth, differentiable mapping with non-zero Jacobian everywhere, enabling gradient-based backpropagation through the output layer of the neural network.

3 SGD

4. **[3 points]**

Consider an objective function comprised of $N = 2$ terms:

$$f(w) = \frac{1}{2}(w-2)^2 + \frac{1}{2}(w+1)^2 \tag{30}$$

Now consider using SGD (with a batch-size $B = 1$) to minimize this objective. Specifically, in each iteration, we will pick one of the two terms (uniformly at random), and take a step in the direction of the negative gradient, with a constant step-size of η.

You can assume η is small enough that every update does result in improvement (aka descent) on the sampled term.

Is SGD guaranteed to decrease the overall loss function in every iteration? If yes, provide a proof. If no, provide a counter-example.

Solution: No. SGD is **not** guaranteed to decrease the overall loss function $f(w)$ in every single iteration.

Proof by Counter-Example:

Let the two individual terms of the objective function be defined as:

$$f_1(w) = \frac{1}{2}(w-2)^2, \quad f_2(w) = \frac{1}{2}(w+1)^2 \tag{31}$$

The total objective function is:

$$f(w) = f_1(w) + f_2(w) = \frac{1}{2}(w^2 - 4w + 4) + \frac{1}{2}(w^2 + 2w + 1) \tag{32}$$

$$= w^2 - w + 2.5 \tag{33}$$

$$= \left(w - \frac{1}{2}\right)^2 + 2.25 \tag{34}$$

The true global minimizer of $f(w)$ is at $w^* = 0.5$, with minimum objective value $f(w^*) = 2.25$.

Now, if we consider starting at the global minimizer $w_0 = 0.5$. The initial overall loss is $f(w_0) = 2.25$.

Suppose term $f_1(w)$ is sampled in the current iteration (which occurs with probability $p = 0.5$). The gradient of the sampled term at $w_0 = 0.5$ is:

$$\nabla f_1(w_0) = w_0 - 2 = 0.5 - 2 = -1.5 \tag{35}$$

The SGD update rule with constant step size $\eta > 0$ yields:

$$w_1 = w_0 - \eta \nabla f_1(w_0) = 0.5 - \eta(-1.5) = 0.5 + 1.5\eta \tag{36}$$

While this update strictly improves the sampled term f_1 (for any $0 < \eta < 2$), on evaluating the new **overall** loss $f(w_1)$:

$$f(w_1) = (w_1 - 0.5)^2 + 2.25 \tag{37}$$

$$= (0.5 + 1.5\eta - 0.5)^2 + 2.25 \tag{38}$$

$$= 2.25\eta^2 + 2.25 \tag{39}$$

For any positive learning rate $\eta > 0$:

$$f(w_1) = 2.25 + 2.25\eta^2 > 2.25 = f(w_0) \tag{40}$$

Thus, the overall loss $f(w)$ given our starting point **strictly increases** after the SGD step.

Conclusion: Because SGD samples a noisy stochastic gradient from an individual data point rather than the true expected full-batch gradient to average over, an individual update can descend along the sampled term while ascending on the overall loss landscape. Therefore, SGD is not guaranteed to decrease the overall loss in every single iteration.

4 Directed Acyclic Graphs (DAG)

One important property for a feed-forward network, as discussed in the lectures, is that it must be a directed acyclic graph (DAG). Recall that a *DAG is a directed graph that contains no directed cycles*. We will study some of its properties in this question.
Let's define a graph $G = (V, E)$ in which V is the set of all nodes as $\{v_1, v_2, ..., v_i, ...v_n\}$ and E is the set of edges $E \subseteq \{(v_i, v_j) \mid v_i, v_j \in V,\ i \neq j\}$.
A *topological order of a directed graph* $G = (V, E)$ is an ordering of its nodes as $\{v_1, v_2, ..., v_i, ...v_n\}$ so that for every edge (v_i, v_j) we have $i < j$.
There are several lemmas that can be inferred from the definition of a DAG. One lemma is: if G is a DAG, then G has a node with no incoming edges.

5. **[3 points]**

   Prove that if the graph G is a DAG, then G has a topological ordering.

   Solution: Proof by Mathematical Induction on the number of vertices $n = |V|$:

   Base Case ($n = 1$): A graph with a single node $V = \{v_1\}$ and consequently an edge set $E = \emptyset$, which is the empty set, has the trivial ordering with just the one vertex (v_1). Since there are no edges, the condition that $i < j$ for every $(v_i, v_j) \in E$ holds vacuously. Thus, the base case holds.

   Inductive Hypothesis: Assume that every finite DAG with k vertices (where $k \geq 1$) has a valid topological ordering.

   Inductive Step: Let $G = (V, E)$ be a DAG with $n = k + 1$ vertices.

   (a) By the provided lemma, every finite DAG has at least one source vertex (a node with in-degree 0). Let $u \in V$ be such a vertex with in-deg$(u) = 0$, meaning there are no incoming edges $(w, u) \in E$ for any $w \in V$.

   (b) We assign u as the first element in our topological ordering: $v_1 = u$.

   (c) Consider the induced subgraph $G' = (V', E')$ obtained by removing vertex u and all its outgoing edges from G, where $V' = V \setminus \{u\}$ and $E' = E \cap (V' \times V')$.

   (d) Since G contains no directed cycles, removing a vertex and edges cannot create any new directed cycles. Therefore, G' is also a DAG with $|V'| = k$ vertices.

   (e) By the inductive hypothesis, G' has a valid topological ordering of its k vertices, denoted by $(v_2, v_3, \ldots, v_{k+1})$.

   (f) We now construct the full sequence for G as $(v_1, v_2, v_3, \ldots, v_{k+1})$.

   (g) To verify that this is a valid topological ordering for G, consider any directed edge $(v_i, v_j) \in E$:

   - If $i = 1$ (the edge originates from $v_1 = u$), then $j \in \{2, 3, \ldots, k + 1\}$, so $1 < j$ holds. Furthermore, because in-deg$(u) = 0$, there can be no edge with $j = 1$.
   - If $i > 1$ and $j > 1$, then the edge (v_i, v_j) belongs to G'. By the inductive hypothesis on G', we have $i < j$.

   Thus, for every directed edge $(v_i, v_j) \in E$, the condition $i < j$ holds.

   By mathematical induction, every finite DAG has a valid topological ordering. ■

6. **[3 points]**

Prove that if the graph G has a topological order, then G is a DAG.

Solution: Proof by Contradiction:

Let $G = (V, E)$ be a directed graph that possesses a topological ordering $(v_1, v_2, \ldots, v_n)$. By definition, for every directed edge $(v_i, v_j) \in E$, the indices satisfy $i < j$. Let $\pi : V \to \{1, 2, \ldots, n\}$ be the bijective mapping giving the position of a vertex in the topological order: $\pi(v_i) = i$. Thus, for every $(u, v) \in E$, we have $\pi(u) < \pi(v)$.

For the sake of contradiction, assume G is **not** a DAG. Then G must contain at least one directed cycle C. Let this directed cycle be represented by a sequence of $m \geq 2$ distinct vertices:

$$C = (u_1, u_2, u_3, \ldots, u_m, u_1) \tag{41}$$

where the edges along the cycle are $(u_1, u_2), (u_2, u_3), \ldots, (u_{m-1}, u_m), (u_m, u_1) \in E$.

Applying the topological ordering property to each successive directed edge in the cycle:

$$(u_1, u_2) \in E \implies \pi(u_1) < \pi(u_2) \tag{42}$$
$$(u_2, u_3) \in E \implies \pi(u_2) < \pi(u_3) \tag{43}$$
$$\vdots \tag{44}$$
$$(u_{m-1}, u_m) \in E \implies \pi(u_{m-1}) < \pi(u_m) \tag{45}$$

By the transitive property of the strict inequality relation $<$ on the integers:

$$\pi(u_1) < \pi(u_2) < \cdots < \pi(u_m) \implies \pi(u_1) < \pi(u_m) \tag{46}$$

However, the cycle also contains the directed edge that closes the cycle: $(u_m, u_1) \in E$. By the definition of the topological ordering, this edge requires:

$$\pi(u_m) < \pi(u_1) \tag{47}$$

Combining these two inequalities gives:

$$\pi(u_1) < \pi(u_m) < \pi(u_1) \implies \pi(u_1) < \pi(u_1) \tag{48}$$

This is a direct contradiction, as the mapped integer for for $\pi(u_1)$ can **NOT** be strictly less than itself ($\pi(u_1) \not< \pi(u_1)$).

Therefore, our initial assumption must be false: G cannot contain any directed cycles and must be a DAG.

5 Automatic Differentiation

7. **[2 points]**

In practice, writing the closed-form expression of the derivative of a loss function f w.r.t. the parameters of a deep neural network is hard (and mostly unnecessary) as f becomes complex. Instead, we define computation graphs and use the automatic differentiation algorithms (typically backpropagation) to compute gradients using the chain rule. For example, consider the expression

$$f(x, y) = (x + y)(y + 1) \tag{49}$$

Let's define intermediate variables a and b such that

$$a = x + y \tag{50}$$

$$b = y + 1 \tag{51}$$

$$f = a \times b \tag{52}$$

A computation graph for the "forward pass" through f is shown in Fig. 1.

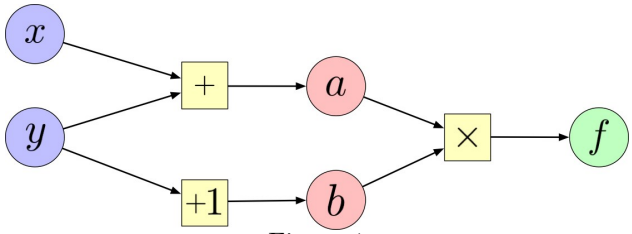


Figure 1

We can then work backwards and compute the derivative of f w.r.t. each intermediate variable ($\frac{\partial f}{\partial a}$, $\frac{\partial f}{\partial b}$) and chain them together to get $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$.

Let $\sigma(\cdot)$ denote the standard sigmoid function. Now, for the following vector function:

$$f_1(w_1, w_2) = \cos(w_1)\cos(w_2) + \sigma(w_2) \tag{53}$$

$$f_2(w_1, w_2) = \ln(w_1 + w_2) + w_1^2 w_2 \tag{54}$$

(a) Draw the computation graph (you might find draw.io useful for this). Compute the value of f at $\mathbf{w} = (1, 2)$.

(b) At this $\mathbf{w}$, compute the Jacobian $\frac{\partial \mathbf{f}}{\partial \mathbf{w}}$ using numerical differentiation (using $\Delta w = 0.01$).

(c) At this $\mathbf{w}$, compute the Jacobian using forward mode auto-differentiation.

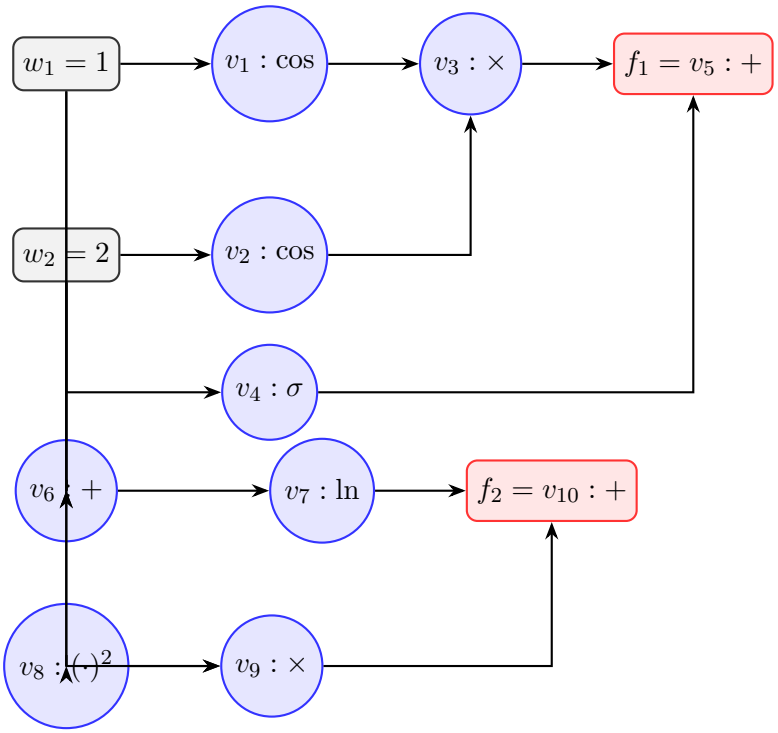
(d) At this $\mathbf{w}$, compute the Jacobian using backward mode auto-differentiation.

(e) Don't you love that software exists to do this for us? *ungraded :)*

Solution: (a) Computation Graph and Evaluation of $\mathbf{f}(1,2)$:

We define the elementary intermediate variables in the computation graph as follows:

$$
\begin{aligned}
v_{-1} &= w_1 = 1 \\
v_0 &= w_2 = 2 \\
v_1 &= \cos(v_{-1}) = \cos(1) \approx 0.540302 \\
v_2 &= \cos(v_0) = \cos(2) \approx -0.416147 \\
v_3 &= v_1 \times v_2 = \cos(1)\cos(2) \approx -0.224845 \\
v_4 &= \sigma(v_0) = \frac{1}{1+e^{-2}} \approx 0.880797 \\
v_5 &= v_3 + v_4 = f_1(w_1, w_2) \approx 0.655952 \\
v_6 &= v_{-1} + v_0 = w_1 + w_2 = 3 \\
v_7 &= \ln(v_6) = \ln(3) \approx 1.098612 \\
v_8 &= (v_{-1})^2 = w_1^2 = 1 \\
v_9 &= v_8 \times v_0 = w_1^2 w_2 = 2 \\
v_{10} &= v_7 + v_9 = f_2(w_1, w_2) \approx 3.098612
\end{aligned}
$$



Evaluating the vector function at $\mathbf{w} = (1, 2)$:

$$
\mathbf{f}(1,2) = \begin{bmatrix} f_1(1,2) \\ f_2(1,2) \end{bmatrix} = \begin{bmatrix} v_5 \\ v_{10} \end{bmatrix} = \begin{bmatrix} \cos(1)\cos(2) + \sigma(2) \\ \ln(3) + 1^2 \cdot 2 \end{bmatrix} \approx \begin{bmatrix} 0.655952 \\ 3.098612 \end{bmatrix} \tag{55}
$$

(b) Numerical Differentiation ($\Delta w = 0.01$):

Using forward finite differences with step $\Delta w = 0.01$:

$$\begin{aligned}
&\mathbf{w} + \Delta w \mathbf{e}_1 = (1.01, 2) \\
&f_1(1.01, 2) = \cos(1.01)\cos(2) + \sigma(2) \approx 0.531843 \times (-0.416147) + 0.880797 = 0.659472 \\
&f_2(1.01, 2) = \ln(3.01) + (1.01)^2 \cdot 2 \approx 1.101940 + 2.040200 = 3.142140 \\
&\mathbf{w} + \Delta w \mathbf{e}_2 = (1, 2.01) \\
&f_1(1, 2.01) = \cos(1)\cos(2.01) + \sigma(2.01) \approx 0.540302 \times (-0.425239) + 0.881845 = 0.652079 \\
&f_2(1, 2.01) = \ln(3.01) + 1^2 \cdot 2.01 \approx 1.101940 + 2.010000 = 3.111940
\end{aligned}$$

Computing each finite-difference quotient:

$$\frac{\partial f_1}{\partial w_1} \approx \frac{f_1(1.01, 2) - f_1(1, 2)}{0.01} = \frac{0.659472 - 0.655952}{0.01} \approx 0.3520 \tag{56}$$

$$\frac{\partial f_1}{\partial w_2} \approx \frac{f_1(1, 2.01) - f_1(1, 2)}{0.01} = \frac{0.652079 - 0.655952}{0.01} \approx -0.3873 \tag{57}$$

$$\frac{\partial f_2}{\partial w_1} \approx \frac{f_2(1.01, 2) - f_2(1, 2)}{0.01} = \frac{3.142140 - 3.098612}{0.01} \approx 4.3528 \tag{58}$$

$$\frac{\partial f_2}{\partial w_2} \approx \frac{f_2(1, 2.01) - f_2(1, 2)}{0.01} = \frac{3.111940 - 3.098612}{0.01} \approx 1.3328 \tag{59}$$

Thus, the numerically estimated Jacobian is:

$$\frac{\partial \mathbf{f}}{\partial \mathbf{w}} \approx \begin{bmatrix} 0.3520 & -0.3873 \\ 4.3528 & 1.3328 \end{bmatrix} \tag{60}$$

(c) Forward-Mode Automatic Differentiation:

Forward mode propagates primal values v_i alongside tangent values $\dot{v}_i = \nabla v_i \cdot \dot{\mathbf{w}}$.

- **Seed 1:** $\dot{\mathbf{w}} = [1, 0]^T$ **($\dot{w}_1 = 1, \dot{w}_2 = 0$):**

$$\begin{aligned}
\dot{v}_1 &= -\sin(v_{-1})\dot{v}_{-1} = -\sin(1)(1) \approx -0.841471 \\
\dot{v}_2 &= -\sin(v_0)\dot{v}_0 = -\sin(2)(0) = 0 \\
\dot{v}_3 &= \dot{v}_1 v_2 + v_1 \dot{v}_2 = (-0.841471)(-0.416147) + 0 \approx 0.350176 \\
\dot{v}_4 &= \sigma(v_0)(1 - \sigma(v_0))\dot{v}_0 = 0 \\
\dot{v}_5 &= \dot{v}_3 + \dot{v}_4 = 0.350176 + 0 = 0.350176 \implies \frac{\partial f_1}{\partial w_1} = 0.350176 \\
\dot{v}_6 &= \dot{v}_{-1} + \dot{v}_0 = 1 + 0 = 1 \\
\dot{v}_7 &= \frac{\dot{v}_6}{v_6} = \frac{1}{3} \approx 0.333333 \\
\dot{v}_8 &= 2v_{-1}\dot{v}_{-1} = 2(1)(1) = 2 \\
\dot{v}_9 &= \dot{v}_8 v_0 + v_8 \dot{v}_0 = (2)(2) + 0 = 4 \\
\dot{v}_{10} &= \dot{v}_7 + \dot{v}_9 = \frac{1}{3} + 4 = \frac{13}{3} \approx 4.333333 \implies \frac{\partial f_2}{\partial w_1} = 4.333333
\end{aligned}$$

- **Seed 2:** $\dot{\mathbf{w}} = [0,1]^T$ **(**$\dot{w}_1 = 0, \dot{w}_2 = 1$**):**

$$\dot{v}_1 = -\sin(1)(0) = 0$$
$$\dot{v}_2 = -\sin(v_0)\dot{v}_0 = -\sin(2)(1) \approx -0.909297$$
$$\dot{v}_3 = \dot{v}_1 v_2 + v_1 \dot{v}_2 = 0 + (0.540302)(-0.909297) \approx -0.491295$$
$$\dot{v}_4 = \sigma(2)(1-\sigma(2))(1) = (0.880797)(0.119203) \approx 0.104994$$
$$\dot{v}_5 = \dot{v}_3 + \dot{v}_4 = -0.491295 + 0.104994 = -0.386301 \implies \frac{\partial f_1}{\partial w_2} = -0.386301$$
$$\dot{v}_6 = \dot{v}_{-1} + \dot{v}_0 = 0 + 1 = 1$$
$$\dot{v}_7 = \frac{\dot{v}_6}{v_6} = \frac{1}{3} \approx 0.333333$$
$$\dot{v}_8 = 2v_{-1}\dot{v}_{-1} = 2(1)(0) = 0$$
$$\dot{v}_9 = \dot{v}_8 v_0 + v_8 \dot{v}_0 = 0 + (1)(1) = 1$$
$$\dot{v}_{10} = \dot{v}_7 + \dot{v}_9 = \frac{1}{3} + 1 = \frac{4}{3} \approx 1.333333 \implies \frac{\partial f_2}{\partial w_2} = 1.333333$$

Thus, the Jacobian computed by forward-mode autodiff is:

$$\frac{\partial \mathbf{f}}{\partial \mathbf{w}} = \begin{bmatrix} 0.350176 & -0.386301 \\ 4.333333 & 1.333333 \end{bmatrix} \tag{61}$$

(d) Backward-Mode Automatic Differentiation:

Backward-mode propagates adjoints $\bar{v}_i = \frac{\partial f_k}{\partial v_i}$ in reverse topological order.

- **Adjoint Seed 1 for** f_1 **(**$\bar{v}_5 = 1, \bar{v}_{10} = 0$**):**

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = 1, \quad \bar{v}_4 = \bar{v}_5 \frac{\partial v_5}{\partial v_4} = 1$$
$$\bar{v}_1 = \bar{v}_3 \frac{\partial v_3}{\partial v_1} = 1 \cdot v_2 = \cos(2) \approx -0.416147$$
$$\bar{v}_2 = \bar{v}_3 \frac{\partial v_3}{\partial v_2} = 1 \cdot v_1 = \cos(1) \approx 0.540302$$
$$\bar{w}_1 = \bar{v}_1 \frac{\partial v_1}{\partial w_1} = (-0.416147)(-\sin(1)) \approx 0.350176$$
$$\bar{w}_2 = \bar{v}_2 \frac{\partial v_2}{\partial w_2} + \bar{v}_4 \frac{\partial v_4}{\partial w_2} = (0.540302)(-\sin(2)) + 1 \cdot \sigma(2)(1-\sigma(2)) \approx -0.491295 + 0.104994 = -0.386301$$

- **Adjoint Seed 2 for** f_2 **(**$\bar{v}_5 = 0, \bar{v}_{10} = 1$**):**

$$\bar{v}_7 = \bar{v}_{10} \frac{\partial v_{10}}{\partial v_7} = 1, \quad \bar{v}_9 = \bar{v}_{10} \frac{\partial v_{10}}{\partial v_9} = 1$$
$$\bar{v}_6 = \bar{v}_7 \frac{\partial v_7}{\partial v_6} = \frac{1}{v_6} = \frac{1}{3}$$
$$\bar{v}_8 = \bar{v}_9 \frac{\partial v_9}{\partial v_8} = 1 \cdot v_0 = 2$$
$$\bar{w}_1 = \bar{v}_6 \frac{\partial v_6}{\partial w_1} + \bar{v}_8 \frac{\partial v_8}{\partial w_1} = \frac{1}{3}(1) + 2(2w_1) = \frac{1}{3} + 4 = \frac{13}{3} \approx 4.333333$$
$$\bar{w}_2 = \bar{v}_6 \frac{\partial v_6}{\partial w_2} + \bar{v}_9 \frac{\partial v_9}{\partial w_2} = \frac{1}{3}(1) + 1(v_8) = \frac{1}{3} + 1 = \frac{4}{3} \approx 1.333333$$

Thus, backward-mode autodiff produces the exact same Jacobian:

$$\frac{\partial \mathbf{f}}{\partial \mathbf{w}} = \begin{bmatrix} 0.350176 & -0.386301 \\ 4.333333 & 1.333333 \end{bmatrix} \tag{62}$$

(e): YES!!! Glad we don't have to implement this manually everytime. Hail PyTorch!

6 Paper Review

The first of our paper reviews for this class comes from a NeurIPS 2019 paper on the topic **'Weight Agnostic Neural Networks'** by **Adam Gaier** and **David Ha** from Google Brain.
The paper presents an interesting proposition that, through a series of experiments, re-examines some fundamental notions about neural networks - in particular, the comparative importance of architectures and weights in a network's predictive performance.
The paper can be viewed here. There's also a helpful interactive webpage with intuitive visualizations to help you understand its key concepts better.
The evaluation rubric for this section is as follows:

8. **[2 points]** Briefly summarize the key contributions, strengths and weaknesses of this paper.

   Solution: **Key Contributions - Summary:** Gaier and Ha challenge the dominant paradigm that neural network capability stems purely from explicit weight optimization via gradient descent. They introduce *Weight Agnostic Neural Networks* (WANNs), architectures discovered through topological search (an extension of NEAT) with varied activation functions that perform reinforcement learning tasks (e.g., CartPole, BipedalWalker, CarRacing) and classification (MNIST) using a single, shared, randomly assigned weight value drawn from a discrete range $[-2, 2]$.

   Strengths:

   (a) **Conceptual Insight:** Demonstrates that architectural inductive bias alone can encode functional behaviors prior to any gradient-based weight training, drawing a compelling parallel to innate biological instincts.

   (b) **Synergy with SGD:** Shows that WANN-discovered topologies serve as superior initializations, achieving competitive performance when fine-tuned with standard optimizers.

   (c) **Simplicity & Thoroughness:** Evaluates diverse RL benchmarks and continuous control tasks across broad weight sweeps with rigorous ablation.

   Weaknesses:

   (a) **Scalability:** Topological evolution, while novel in theory, struggles to scale to high-dimensional state spaces and representations, particularly in the case of modern deep vision/NLP architectures.

   (b) **Performance Ceiling:** While impressive for shared constant weights, raw performance without weight tuning lags behind standard overparameterized networks trained end-to-end with backpropagation.

9. **[2 points]** What is your personal takeaway from this paper? This could be expressed either in terms of relating the approaches adopted in this paper to your traditional understanding of learning parameterized models, or potential future directions of research in the area which the authors haven't addressed, or anything else that struck you as being noteworthy.

   Solution: Personal Takeaway: The central insight and takeaway of WANNs fundamentally shifts how we view the duality of neural architecture search and parameter learning. Traditional deep learning treats architecture as a passive canvas and delegates all representation learning to weight optimization. WANN demonstrates that topology is an active computational inductive bias that encodes strong functional priors mirroring precocial species born with innate motor skills.

   Future Directions & Connections:

   (a) **Modular Macro-Topologies for Vision:** Replacing node-level evolution with modular, motif-based search (e.g., searching connections between fixed convolutional or attention blocks) could scale weight-agnostic priors to complex visual processing without intractable graph growth.

   (b) **Weight-Agnostic Subnetwork Discovery in LLMs:** Integrating WANN concepts with pruning algorithms could identify pre-existing, weight-agnostic routing subnetworks inside large transformer models, enabling low-power inference for natural language understanding.

Guidelines: Please restrict your reviews to no more than 350 words (total length for answers to both the above questions).